6주차 예습과제- 딥러닝 파이토치 교과서 6장-Part 2 (6.1.5~)

6장- 합성공 신경망 II

> 💡층을 깊게 쌓으면 오히려 성능이 떨어지는 문제를 → **입력 x를 출력에 그대로 더해주는 지름길(숏컷)** 로 해결한 모델.
> 층은 전체 답 H(x)가 아니라 **차이 F(x) = H(x) – x 만** 배우면 된다.

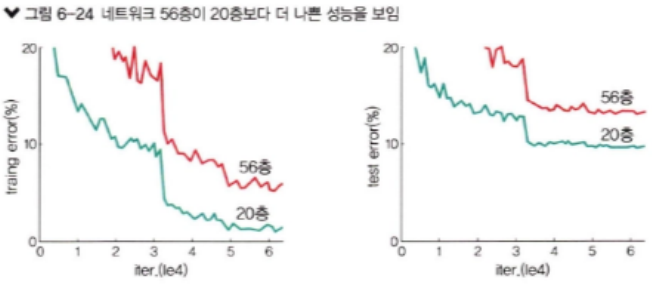
6.1.5 ResNet

1. ResNet이란?

- **마이크로소프트**에서 개발
- 논문: **"Deep Residual Learning for Image Recognition"**
- 핵심: 깊어진 신경망을 효과적으로 학습하기 위해 **레지듀얼(residual)** 개념을 고안

2. 왜 필요한가? — 깊을수록 좋지 않다

- 보통은 신경망이 깊을수록 성능이 좋아질 것 같지만, 실제로는 그렇지 않음
- 논문에 따르면 깊이가 깊어질수록 성능이 좋아지다가 **일정 단계를 넘으면 오히려 나빠짐**
- **그림 6-24**: 56층이 20층보다 더 나쁜 성능



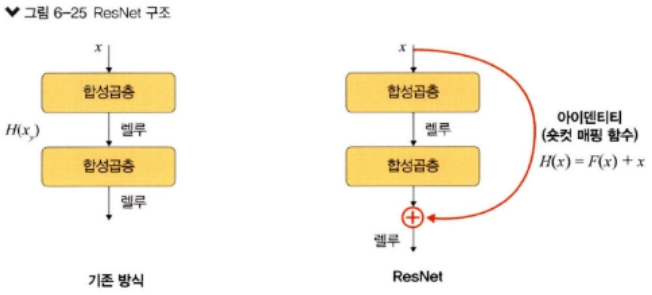
그래프	20층	56층	해석
training error(%)	더 낮음	더 높음	학습 데이터에서도 56층이 더 나쁨
test error(%)	더 낮음	더 높음	테스트에서도 56층이 더 나쁨
축	x: iter.(1e4) 0~6 / y: error 0~20%		깊다고 무조건 좋아지지 않음

> 🔎 학습 오차(training error)까지 더 나쁘다는 건 과적합 때문이 아니라는 뜻. 학습 자체가 잘 안 되는 것
> → 이를 **성능 저하(degradation) 문제**라고 부름.

- 해결책: **레지듀얼 블록(residual block)** 도입
  - 기울기가 잘 전파되도록 일종의 **숏컷(shortcut, skip connection)** 을 만들어 줌
- 이 개념이 필요한 이유

- 2014년 **GoogLeNet: 22층** vs **ResNet: 152층**
- 층이 이렇게 많으면 **기울기 소멸 문제** 발생 가능 → 숏컷으로 방지

3. 기본 구조 — 기존 방식 vs ResNet

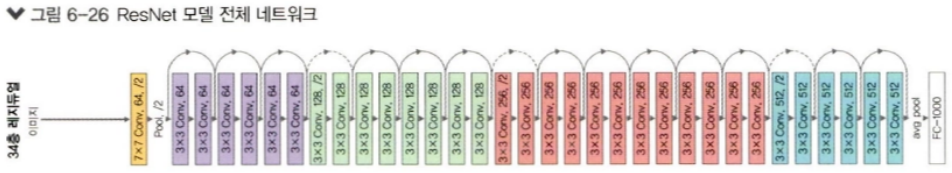


구분	기존 방식	ResNet
흐름	x → 합성곱층 → 렐루 → 합성곱층 → 렐루	x → 합성곱층 → 렐루 → 합성곱층 → ⊕(+x) → 렐루
층이 학습하는 것	출력 H(x) 전체	잔차 F(x) = H(x) − x (차이만)
출력 식	H(x)	H(x) = F(x) + x
지름길(숏컷)	없음	아이덴티티(숏컷 매핑 함수)로 x를 그대로 전달
층이 깊어질 때	기울기 소멸 → 성능 저하	숏컷으로 기울기가 잘 전파됨

> 🔎 **왜 기울기가 잘 흐를까?**
> H(x) = F(x) + x 를 x로 미분하면 ∂H/∂x = ∂F/∂x + 1
> 항상 **+1** 이 붙기 때문에, F 쪽 기울기가 아무리 작아져도 숏컷을 통해 기울기가 0이 되지 않고 앞층까지 전달됨 (기울기의 고속도로).

4. 블록(block)과 레지듀얼 블록

- **블록** = 계층의 묶음
  - 엄밀히는 **합성곱층을 하나의 블록으로 묶은 것**
- 그림 6-26에서 색상별(보라색, 노란색 등)로 블록을 구분
- 이렇게 묶인 계층들 = **레지듀얼 블록(residual block)**
- 레지듀얼 블록을 여러 개 쌓은 것 = **ResNet**



구간	층 구성	개수	비고
시작	7×7 Conv, 64, /2	1	stride 2
풀링	Pool, /2	–	크기 절반

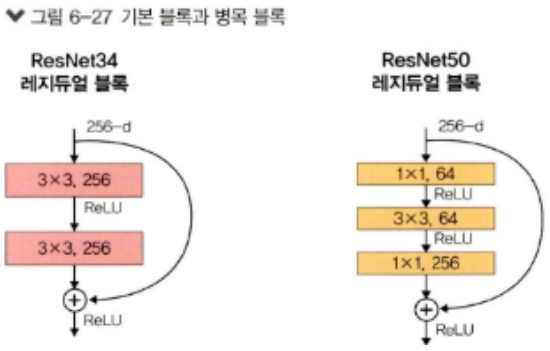
구간	층 구성	개수	비고
보라색	3×3 Conv, 64	6 (블록 3개)	아이덴티티 숏컷
초록색	3×3 Conv, 128 (첫 층 /2)	8 (블록 4개)	첫 블록은 점선 숏컷(다운샘플)
빨간색	3×3 Conv, 256 (첫 층 /2)	12 (블록 6개)	첫 블록은 점선 숏컷(다운샘플)
파란색	3×3 Conv, 512 (첫 층 /2)	6 (블록 3개)	첫 블록은 점선 숏컷(다운샘플)
끝	avg pool → FC 1000	1 (FC)	분류
합계	1 + 32 + 1 = 34층	블록 16개	3 + 4 + 6 + 3 = 16

5. 병목 블록(bottleneck block)

5-1. 문제: 파라미터 폭증

- 계층을 계속 쌓으면 **파라미터 수**가 문제가 됨 → 깊어질수록 파라미터 증가
- 예) **ResNet34**: 합성곱층 **34개**, 블록 **16개**
  - 첫 번째 블록 파라미터가 **1152K**라면 → 전체 파라미터 수 **2만 1282K**
- 이렇게 깊어질수록 파라미터는 무제한으로 커짐 → 해결책: **병목 블록**

5-2. 기본 블록 vs 병목 블록



구분	기본 블록 (basic block)	병목 블록 (bottleneck block)
사용 모델	ResNet34	ResNet50
입력	256-d	256-d
구조	3×3, 256 → ReLU → 3×3, 256 → ⊕ → ReLU	1×1, 64 → ReLU → 3×3, 64 → ReLU → 1×1, 256 → ⊕ → ReLU
합성곱층 수	2개	3개 (더 깊음)
파라미터 수 (교재 수치)	39.3216M	6.9632M
핵심	3×3을 256채널 그대로 두 번	1×1로 채널 축소(256→64) → 3×3 → 1×1로 다시 확장(64→256)
+ (아이덴티티 매핑)	있음	있음

- ✅ **깊이는 깊어졌는데 파라미터 수는 감소!**

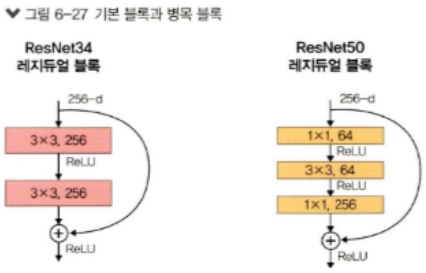
5-3. 어떻게 가능할까?

- ResNet50은 3×3 합성곱층 **앞뒤로 1×1 합성곱층**이 붙어 있음
- **1×1 합성곱층으로 채널 수를 조절** → 차원을 **줄였다가(256→64) 다시 늘림(64→256)**
- 비싼 3×3 연산은 얇은 64채널에서만 수행 → 파라미터 감소
- 가운데가 좁아지는 모양이 **병목(bottleneck)** 같아서 병목 블록이라고 부름

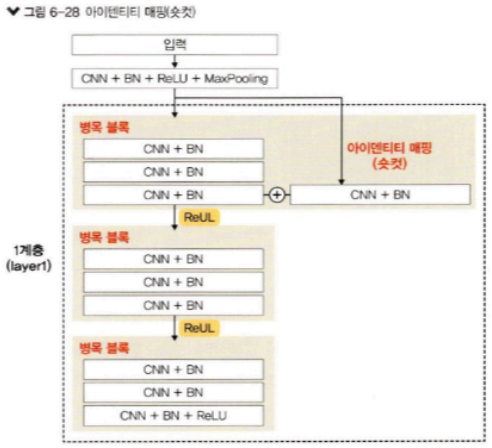
> 🧠 모래시계 모양. 넓은 입구(256) → 좁은 목(64)에서 핵심 작업(3×3) → 다시 넓은 출구(256). 무거운 작업은 좁은 곳에서만 하니까 비용이 확 줄어든다.

6. 아이덴티티 매핑(identity mapping)

- = **숏컷(shortcut)** = **스킵 연결(skip connection)**
- 그림 6-27 아래쪽의 **+ 기호** 부분 (기본 블록·병목 블록 **모두** 사용)



- 정의: 입력 x가 어떤 함수를 통과하더라도 **다시 x라는 형태로 출력**되도록 하는 것
- **그림 6-28 (1계층, layer1)**
- 입력 → CNN + BN + ReLU + MaxPooling
- 병목 블록 ×3 (각 블록: CNN + BN 3개)
  - 첫 번째 병목 블록: 숏컷 쪽에도 **CNN + BN** → ⊕ → ReLU
  - 두 번째 병목 블록: → ReLU
  - 세 번째 병목 블록: 마지막은 CNN + BN + ReLU



6-1. 코드로 보면 (forward 핵심)

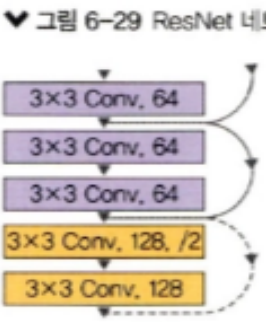
- `i = x` → 입력을 i에 저장
- conv1 → bn1 → relu → conv2 → bn2 통과
- `if self.downsample is not None: i = self.downsample(i)` → 다운샘플 적용
- `x += i` → **아이덴티티 매핑 적용**
- relu 후 return

6-2. 형태(shape) 예시

- x = **(28, 28, 64)** 라고 가정
- i에 저장 → i도 **(28, 28, 64)**
- 합성곱층을 통과한 결과에 같은 형태를 더함 → 최종도 **(28, 28, 64)** 그대로

7. 다운샘플(downsample)

- 또 다른 핵심 개념
- **특성 맵(feature map) 크기를 줄이기 위한 것** → 풀링과 같은 역할
- **그림 6-29 (ResNet 네트워크 일부)**
  - 보라색: 3×3 Conv, 64 (×3)
  - 노란색: 3×3 Conv, 128, **/2** → 3×3 Conv, 128 (점선 숏컷)



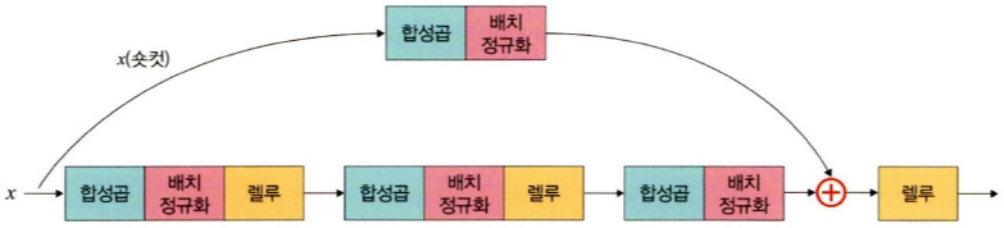
- **보라색 영역**
  - 첫 블록 특성 맵 (28, 28, 64) → 세 번째 블록 마지막 합성곱 + 아이덴티티 매핑까지 끝나도 **(28, 28, 64)**
- **노란색 영역**
  - 시작 지점에서 채널 수가 **128**로 증가
  - **/2** → 첫 블록 합성곱층의 **스트라이드가 2** → **(14, 14, 128)**
- ⚠️ 보라색 (28, 28, 64)과 노란색 (14, 14, 128)은 **형태가 다름**
  - 형태를 맞추지 않으면 **더할 수 없음** → **아이덴티티 매핑 불가**

- 그래서 **숏컷(아이덴티티) 쪽에 다운샘플이 필요**

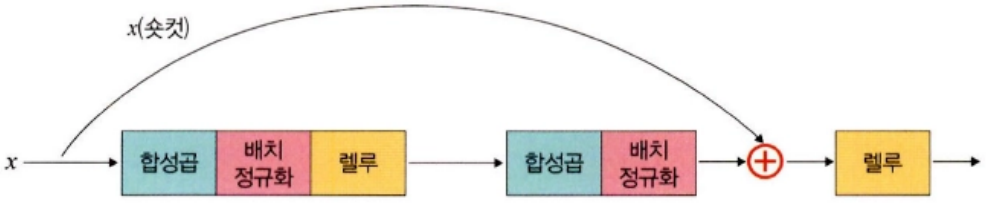
7-1. 형태 맞추는 방법

- 입력과 출력 형태를 같게 맞추려면 → **스트라이드 2를 가진 1×1 합성곱 계층**을 하나 연결

❮ 그림 6-30 합성곱 블록



❮ 그림 6-31 아이덴티티 블록



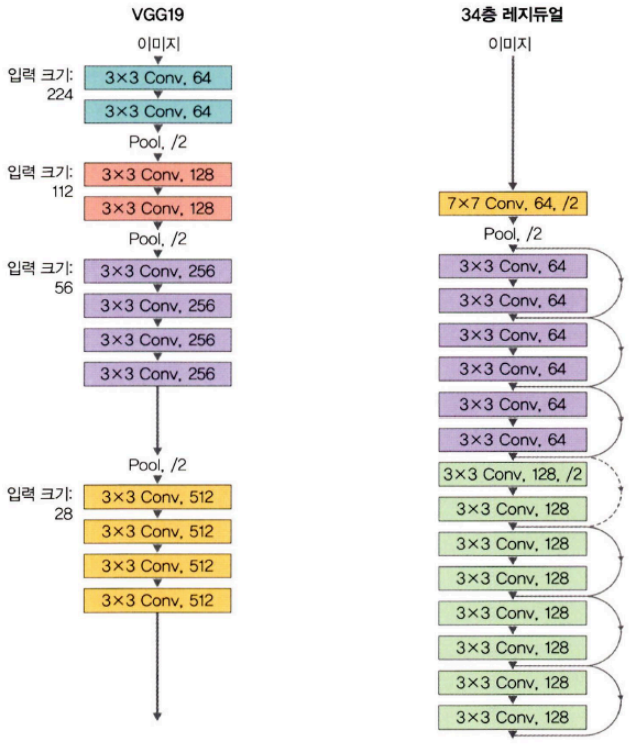
구분	아이덴티티 블록 (identity block)	프로젝션 숏컷 = 합성곱 블록 (projection-shortcut)
그림	6-31	6-30
조건	입력 차원 = 출력 차원	입력 차원 ≠ 출력 차원
숏컷 경로	x를 그대로 전달	x → 합성곱(1×1, stride 2) → 배치 정규화 → 차원 맞춘 뒤 전달
메인 경로 (그림 기준)	합성곱·배치정규화·렐루 → 합성곱·배치정규화 → ⊕ → 렐루	합성곱·배치정규화·렐루 → 합성곱·배치정규화·렐루 → 합성곱·배치정규화 → ⊕ → 렐루
코드	downsample = None	i = self.downsample(i) 적용
예시	(28, 28, 64) → (28, 28, 64)	(28, 28, 64) → (14, 14, 128)

> 🧠 더하기는 같은 모양끼리만 가능. 숏컷 쪽 x를 "같은 규격으로 포장(1×1 conv, stride 2)"해줘야 메인 경로 결과와 합칠 수 있다.

8. 정리 — ResNet = VGG19 + 깊게 + 숏컷

- ResNet은 기본적으로 **VGG19 구조를 뼈대**로 함
- 거기에 **합성곱층을 추가해 깊게** 만든 후 **숏컷들을 추가**한 것이 사실상 전부
- **그림 6-32 (VGG19 vs 34층 레지듀얼)**

❤ 그림 6-32 VGG19와 ResNet 비교



입력 크기	VGG19	34층 레지듀얼
224	3×3 Conv 64 ×2 → Pool /2	7×7 Conv 64, /2 → Pool /2
112	3×3 Conv 128 ×2 → Pool /2	–
56	3×3 Conv 256 ×4 → Pool /2	3×3 Conv 64 ×6 (숏컷 3개)
28	3×3 Conv 512 ×4	3×3 Conv 128 ×8 (첫 층 /2, 숏컷 4개)
차이	숏컷 없음, 풀링으로 크기 축소	숏컷 있음, stride 2 합성곱(/2)으로 크기 축소

6.1.5 ResNet — 파이토치 구현

> 💡데이터 준비(전처리 → 데이터셋 → 데이터로더) → **기본 블록 / 병목 블록** 정의 → 블록을 쌓는 **ResNet 클래스** → **Config(네임드튜플)** 만 바꿔서 ResNet18~152를 찍어냄.
> 직접 만든 ResNet50과 torchvision의 사전 훈련된 ResNet50 구조가 같은지 비교.

Overall Steps

단계	코드	하는 일
1	6-65	라이브러리 호출, device 설정
2	6-66, 6-67	이미지 전처리 클래스 + 전처리 변수값 정의
3	6-68, 6-69	개/고양이 이미지 경로 불러오기 → 훈련·검증·테스트 분리
4	6-70, 6-71	커스텀 Dataset (레이블 구분) → 데이터셋 생성
5	6-72	DataLoader로 배치 단위로 메모리에 불러오기
6	6-73	기본 블록(BasicBlock) 정의
7	6-74	병목 블록(Bottleneck) 정의

단계	코드	하는 일
8	6-75	ResNet 전체 네트워크 정의
9	6-76 ~ 6-78	ResNetConfig로 ResNet18/34/50/101/152 설정
10	6-79, 6-80	사전 훈련된 ResNet50 불러와서 구조 출력
11	6-81	직접 만든 ResNet50 생성 → 구조 비교

1. 라이브러리 호출 (코드 6-65)

- 파이토치: `torch`, `torch.nn as nn`, `torch.nn.functional as F`, `torch.optim as optim`, `torch.utils.data as data`
- torchvision: `torchvision`, `transforms`, `datasets`, `models`
- 시각화·연산: `matplotlib.pyplot as plt`, `numpy as np`
- 기타: `copy`, `namedtuple`(collections), `os`, `random`, `time`
- 이미지: `cv2`, `PIL.Image`
- 데이터: `DataLoader`, `Dataset`
- 핵심 한 줄: `device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')` → GPU 있으면 GPU, 없으면 CPU

네임드튜플(namedtuple)

- 파이썬 자료형 중 하나
- **튜플의 성질**을 가지면서 → **인덱스뿐 아니라 키 값으로도** 데이터에 접근 가능

코드	의미	결과
`Student = namedtuple('Student', ['name','age','DOB'])`	네임드튜플 정의	–
`S = Student('홍길동', '19', '187')`	값 추가	–
`S[1]`	인덱스로 접근	19
`S.name`	키 값으로 접근	홍길동

> 🧠 일반 튜플은 "몇 번째 칸"으로만 찾는 사물함, 네임드튜플은 **칸마다 이름표가 붙은 사물함**. 번호로도, 이름으로도 꺼낼 수 있음.
> → 뒤에서 ResNet 설정값(block, n_blocks, channels)을 묶는 데 사용됨

2. 이미지 데이터 전처리 (코드 6-66, 6-67)

- 각 파라미터는 앞에서 살펴봤으므로 설명 생략 (교재)
- `ImageTransform` 클래스: `__init__(resize, mean, std)`에서 train/val 전처리를 딕셔너리로 정의, `__call__(img, phase)`에서 phase에 맞는 전처리 적용

구분	'train' (훈련 이미지)	'val' (검증·테스트 이미지)
크기 조정	RandomResizedCrop(resize, scale=(0.5, 1.0))	Resize(256) → CenterCrop(resize)
뒤집기	RandomHorizontalFlip()	없음

구분	'train' (훈련 이미지)	'val' (검증·테스트 이미지)
텐서 변환	ToTensor()	ToTensor()
정규화	Normalize(mean, std)	Normalize(mean, std)
목적	랜덤 변형으로 데이터 증강	매번 같은 결과가 나오도록 고정된 변환

전처리 변수값 (코드 6-67)

변수	값
size	224
mean	(0.485, 0.456, 0.406)
std	(0.229, 0.224, 0.225)
batch_size	32

> 🔎 mean·std 값은 ImageNet 데이터셋의 RGB 채널별 평균·표준편차. 사전 훈련 모델들이 이 값으로 학습돼서 관례적으로 그대로 씀.

---

3. 데이터셋 불러오기 & 분리 (코드 6-68, 6-69)

- `torchvision.datasets.ImageFolder`를 이용해 훈련·테스트 데이터셋을 불러옴
- 데이터셋: 계속 써온 **개와 고양이** 예제 (캐글 데이터나 책의 데이터셋 사용 가능)
- 경로: `../chap06/data/dogs-vs-cats/Cat/`, `.../Dog/`
- 과정
  1. `os.listdir`로 각 폴더의 파일 경로를 모아 `sorted`
  2. 고양이 + 개 경로를 하나의 리스트로 합침 (`[*cat, *dog]`)
  3. `cv2.imread(i) is not None`인 것만 남김 → **읽을 수 없는 깨진 이미지 제거**
- 분리: `random.seed(42)` → `random.shuffle` 후 슬라이싱

용도	슬라이싱	개수
훈련	`[:400]`	400
검증	`[400:-10]`	92
테스트	`[-10:]`	10

---

4. 커스텀 데이터셋 & 레이블 (코드 6-70, 6-71)

- `DogvsCatDataset(Dataset)`: 이미지를 가져와 전처리 적용 + 레이블 부여

메서드	하는 일
`__init__(file_list, transform=None, phase='train')`	파일 목록, 전처리, phase 저장
`__len__`	전체 이미지 수 반환
`__getitem__(idx)`	이미지 열기(Image.open) → 전처리(transform(img, phase)) → 레이블 반환

- 레이블: 파일 경로에서 이름 추출 `img_path.split('/')[-1].split('.')[0]`
  - **dog → 1, cat → 0**
- `train_dataset`(phase='train'), `val_dataset`(phase='val') 생성
- index 0 출력 결과

출력	값	의미
`train_dataset.__getitem__(0)[0].size()`	torch.Size([3, 224, 224])	컬러(채널 3), 224×224
`train_dataset.__getitem__(0)[1]`	0	고양이

5. 데이터로더 (코드 6-72)

- 데이터로더로 데이터를 메모리에 불러옴 → **배치 크기만큼 나눠서** 불러옴

이터레이터	설정
train_iterator	DataLoader(train_dataset, batch_size=32, shuffle=True)
valid_iterator	DataLoader(val_dataset, batch_size=32, shuffle=False)
dataloader_dict	{'train': train_iterator, 'val': valid_iterator}

- `next(iter(train_iterator))`로 한 배치 확인
  - 이미지: **torch.Size([32, 3, 224, 224])** → (배치, 채널, 높이, 너비)
  - 레이블: 0/1로 이루어진 길이 32의 텐서

6. 기본 블록 BasicBlock (코드 6-73)

- **ResNet18, ResNet34**에서 사용
- **3×3 합성곱 두 개**로 구성
- `expansion = 1` → 출력 채널 = out_channels 그대로

층	설정
conv1	3×3, in → out, **stride=stride**, padding=1, bias=False
bn1	BatchNorm2d(out)
conv2	3×3, out → out, stride=1, padding=1, bias=False
bn2	BatchNorm2d(out)
relu	ReLU(inplace=True)
downsample (True일 때)	Sequential(1×1 conv in → out, stride=stride + BN)
downsample (False일 때)	None

- forward: `i = x` → conv1 → bn1 → relu → conv2 → bn2 → (downsample 있으면 `i = self.downsample(i)`) → `x += i` → relu

① 다운샘플이 적용되는 부분

- 입력 데이터 크기와, 네트워크를 통과한 후 **출력 데이터 크기가 다를 경우** 사용

- 다운샘플을 위해 합성곱층에 **스트라이드**를 적용
- 교재 예시: `nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=1, bias=False)`
  - 예제에서는 stride=stride로 되어 있지만, 클래스 속성으로 stride=1이 정의됐기 때문에 stride=1로 작성했다고 함

② 아이덴티티 매핑이 적용되는 부분 (`x += i`)

- 특정 층의 출력 결과를 → 다음 합성곱층을 통과한 출력 결과에 **더해줌** → 그래서 **스킵 연결(skip connection)** 이라고도 함
- 예: x에 conv1, bn1, relu, conv2, bn2, relu, conv3, bn3 값이 더해지다가 **초기의 x가 다시 더해지는 것** = 아이덴티티 매핑 = 숏컷 = 스킵 연결

> 🧠 `i = x`는 원본을 복사해 두는 것. 가공(conv·bn)을 다 끝낸 뒤 마지막에 원본을 다시 얹는 게 `x += i`. 원본과 가공본의 크기가 다르면 원본도 같은 규격으로 줄여야(downsample) 얹을 수 있음.

---

7. 병목 블록 Bottleneck (코드 6-74)

- **ResNet50, ResNet101, ResNet152**에서 사용
- **1×1 합성곱 → 3×3 합성곱 → 1×1 합성곱**으로 구성
- `expansion = 4` → ResNet에서 병목 블록을 정의하기 위한 하이퍼파라미터 (출력 채널을 4배로 늘림)

층	설정	역할
conv1	1×1, in → out, stride=1, bias=False	채널 축소
bn1	BatchNorm2d(out)	–
conv2	3×3, out → out, **stride=stride**, padding=1, bias=False	핵심 연산 (크기 축소도 여기서)
bn2	BatchNorm2d(out)	–
conv3	1×1, out → **expansion × out**, stride=1, bias=False	채널 확장 (다음 계층의 입력 채널 수와 일치시키기 위해)
bn3	BatchNorm2d(expansion × out)	–
relu	ReLU(inplace=True)	–
downsample (True 일 때)	Sequential(1×1 conv in → expansion × out, stride=stride + BN)	숏컷 규격 맞추기

- forward: `i = x` → conv1 → bn1 → relu → conv2 → bn2 → relu → conv3 → bn3 → (downsample) → `x += i` → relu

기본 블록 vs 병목 블록 (코드 기준)

구분	기본 블록	병목 블록
사용 모델	ResNet18, ResNet34	ResNet50, ResNet101, ResNet152
구조	3×3 합성곱층 2개	1×1 → 3×3 → 1×1 합성곱층
expansion	1	4

구분	기본 블록	병목 블록
블록 출력 채널	out_channels	4 × out_channels
활성화 함수(ReLU) 수	2	3

- 기본 블록 → 병목 블록으로 바꾸는 이유
  - 계층을 **더 깊게** 쌓으면서도 **계산 비용을 줄일 수 있기 때문**
  - 계층이 많아짐 = 활성화 함수가 더 많이 포함됨 → **더 많은 비선형성(non-linearity)** 처리 가능 → 다양한 입력 데이터 처리 가능
- 결국 **아이덴티티 매핑 + 병목 블록** 덕분에 ResNet은 더욱 깊은 계층을 쌓을 수 있게 됨

---

8. ResNet 전체 네트워크 (코드 6-75)

- ResNet도 VGG처럼 다양한 모델(18, 34, 50, 101, 152)이 있음
- `ResNet(config, output_dim, zero_init_residual=False)`
  - `block, n_blocks, channels = config` → ResNet 호출 시 넘겨준 config 값을 저장
  - `self.in_channels = channels[0]`
  - `assert len(n_blocks) == len(channels) == 4` → 블록 크기 = 채널 크기 = 4 (layer가 4개)

구성 요소

구성	설정
conv1	Conv2d(3 → 64, kernel 7, stride 2, padding 3, bias=False)
bn1, relu	BatchNorm2d(64), ReLU(inplace=True)
maxpool	MaxPool2d(kernel 3, stride 2, padding 1)
layer1	get_resnet_layer(block, n_blocks[0], channels[0]) → stride 1
layer2	get_resnet_layer(..., n_blocks[1], channels[1], stride=2)
layer3	get_resnet_layer(..., n_blocks[2], channels[2], stride=2)
layer4	get_resnet_layer(..., n_blocks[3], channels[3], stride=2)
avgpool	AdaptiveAvgPool2d((1, 1))
fc	Linear(in_channels → output_dim)

get_resnet_layer — 블록을 추가하기 위한 함수

1. `self.in_channels != block.expansion * channels` 이면 → `downsample = True`, 같으면 False
2. **첫 번째 블록**만 `block(in_channels, channels, stride, downsample)` → 계층 추가 시 in_channels, channels, stride뿐 아니라 **다운샘플 적용 유무도 함께 전달**
3. 나머지 `n_blocks - 1`개는 `block(block.expansion * channels, channels)` → stride 1, 다운샘플 없음
4. `self.in_channels = block.expansion * channels` 로 갱신 → 다음 layer의 입력이 됨
5. `nn.Sequential(*layers)` 반환

> 🧠 각 layer는 "입구 블록 1개 + 일반 블록 여러 개". 크기·채널이 바뀌는 건 **입구 블록에서만** 일어나고, 그래서 다운샘플(숏컷 규격 맞추기)도 입구에만 붙음.

forward와 특성 맵 크기

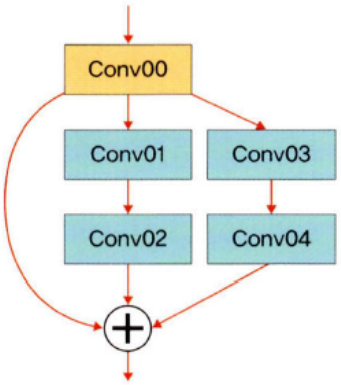
단계	교재 주석 크기
conv1	224×224
maxpool	112×112
layer1	56×56
layer2	28×28
layer3	14×14
layer4	7×7
avgpool	1×1
`h = x.view(x.shape[0], -1)`	평탄화
`x = self.fc(h)`	최종 출력 → `return x, h`

> 🔎 실제 각 층을 **통과한 뒤** 크기는 conv1 후 112 → maxpool 후 56 → layer1 후 56 → layer2 후 28 → layer3 후 14 → layer4 후 7 → avgpool 후 1. 교재 주석은 앞쪽(conv1, maxpool)이 들어가는 입력 크기 기준으로 적혀 있어서 한 칸씩 밀려 보임.

① zero_init_residual

- 각 **레지듀얼 분기(residual branch)** 의 **마지막 BN을 0으로 초기화** → 다음 레지듀얼 분기를 0에서 시작할 수 있도록 함
  - Bottleneck이면 `bn3.weight = 0`, BasicBlock이면 `bn2.weight = 0`
- 모델 생성·학습과는 상관없지만, 논문(arxiv.org/abs/1706.02677)에 따르면 BN을 0으로 초기화하면 성능이 **0.2~0.3%** 정도 향상 → ResNet에서 많이 사용
- **레지듀얼 분기**: 프로그램에서 조건에 따라 A, B, C 등으로 분기하는 것과 같음
- **그림 6-33**: Conv00 → (Conv01 → Conv02) / (Conv03 → Conv04) 두 갈래 + 숏컷 → ⊕ 에서 합쳐짐

그림 6-33 레지듀얼 분기



> 🔎 마지막 BN 가중치가 0이면 F(x) = 0 → 블록 출력 = 0 + x = x. 즉 학습 초기엔 모든 블록이 "그냥 통과"처럼 동작해서 깊은 네트워크도 얕은 네트워크처럼 안정적으로 학습을 시작함.

---

9. ResNetConfig (코드 6-76 ~ 6-78)

- `ResNetConfig = namedtuple('ResNetConfig', ['block', 'n_blocks', 'channels'])`
- 네임드튜플로 block, n_blocks, channels를 묶어서 저장 → 모델별로 값만 바꿔 끼우면 됨

모델	block	n_blocks	channels	층 수 계산 (보충)
ResNet18	BasicBlock	[2, 2, 2, 2]	[64, 128, 256, 512]	8블록 × 2 + 2 = 18
ResNet34	BasicBlock	[3, 4, 6, 3]	[64, 128, 256, 512]	16블록 × 2 + 2 = 34
ResNet50	Bottleneck	[3, 4, 6, 3]	[64, 128, 256, 512]	16블록 × 3 + 2 = 50
ResNet101	Bottleneck	[3, 4, 23, 3]	[64, 128, 256, 512]	33블록 × 3 + 2 = 101
ResNet152	Bottleneck	[3, 8, 36, 3]	[64, 128, 256, 512]	50블록 × 3 + 2 = 152

- ※ "+2" = 맨 앞 7×7 conv1 + 맨 끝 fc
- ResNet34와 ResNet50은 n_blocks가 [3, 4, 6, 3]으로 같고 **블록 종류만 다름**

---

10. 사전 훈련된 ResNet (코드 6-79, 6-80)

- 아이덴티티 매핑, 다운샘플, 병목 블록까지 정의하려니 꽤 번거로움
- ResNet도 사전 훈련된 모델 → **코드 한 줄로** 사용 가능
  - `pretrained_model = models.resnet50(pretrained=True)`

print(pretrained_model) 구조 요약

위치	conv1 (1×1)	conv2 (3×3)	conv3 (1×1)	downsample
시작	Conv2d(3, 64, 7×7, stride 2, padding 3) → BN → ReLU → MaxPool(3, stride 2, padding 1)			
layer1 (0)	64 → 64	64 → 64, stride 1	64 → 256	Conv2d(64, 256, 1×1, stride 1) + BN(256)
layer1 (1)	256 → 64	64 → 64, stride 1	64 → 256	없음
… 중간 생략 …				
layer4 (0)	1024 → 512	512 → 512, **stride 2**	512 → 2048	Conv2d(1024, 2048, 1×1, **stride 2**) + BN(2048)
layer4 (1), (2)	2048 → 512	512 → 512, stride 1	512 → 2048	없음
끝	AdaptiveAvgPool2d(1, 1) → Linear(2048 → 1000)			

- 모든 Conv2d는 bias=False, BN 설정은 eps=1e-05, momentum=0.1

layer	입력 채널	중간(병목) 채널	출력 채널 (×4)	첫 블록 stride	첫 블록 다운샘플
layer1	64	64	256	1	O (64 ≠ 256)
layer2	256	128	512	2	O
layer3	512	256	1024	2	O
layer4	1024	512	2048	2	O

- ResNet34(기본 블록, expansion 1)는 layer1에서 64 = 1 × 64 이므로 **다운샘플 없음**, layer2~4만 다운샘플

---

11. 직접 만든 ResNet50 (코드 6-81)

- Config 부분에 `resnet50_config` 지정, `OUTPUT_DIM = 2` → 테스트 데이터셋의 클래스(개와 고양이) 2개
- `model = ResNet(resnet50_config, OUTPUT_DIM)` → `print(model)`
- 수동으로 작성한 네트워크 구조를 사전 훈련된 ResNet과 비교해볼 것 (깊이가 깊어 교재에서는 중간 생략)
  - conv1 Conv2d(3, 64, 7×7, stride 2, padding 3), bn1, relu, maxpool(3, 2, 1) 동일
  - layer1 (0) Bottleneck: 64 → 64 (1×1), 64 → 64 (3×3), 64 → 256 (1×1), downsample Conv2d(64, 256, 1×1, stride 1) 동일
  - 차이점: 마지막 fc 출력이 1000(ImageNet 클래스) 대신 **2**

직접 만든 모델 vs 사전 훈련된 모델 비교 결과

구분	사전 훈련된 ResNet50	직접 만든 ResNet50
conv1 ~ layer4	Conv2d(3, 64, 7×7, stride 2) ... Bottleneck 구조	동일
avgpool	AdaptiveAvgPool2d(1, 1)	동일
fc	Linear(2048 → **1000**)	Linear(2048 → **2**)

- 결론: 힘들게 정의한 네트워크와 사전 정의된 네트워크는 **다르지 않음**
- 앞으로 ResNet이 필요하면 사전 훈련된 모델을 코드 한두 줄로 불러와 쓰게 될 것
- 하지만 **구조를 알고 쓰는 것과 모르고 쓰는 것은 큰 차이**
  - 네트워크를 변경하거나 입출력 텐서의 크기를 알아야 할 때, 구조를 알면 수정이 어렵지 않음
  - 논문에서 새로운 모델이 발표되더라도 **그 구조부터 익히는 훈련**이 필요

12. 옵티마이저와 손실 함수 (코드 6-82)

항목	설정
옵티마이저	`optim.Adam(model.parameters(), lr=1e-7)` → lr = 1×10^{-7}
손실 함수	`nn.CrossEntropyLoss()`
장치	`model.to(device)`, `criterion.to(device)`

> 🔎 lr = 0.0000001은 매우 작은 학습률. 가중치가 거의 안 움직이니, 뒤의 학습 결과가 50% 근처에서 맴도는 원인 중 하나일 수 있음.

13. 정확도 측정 함수 (코드 6-83)

- `calculate_topk_accuracy(y_pred, y, k=2)`: 모델이 얼마나 잘 학습되었는지 측정
- `torch.no_grad()` 안에서 계산 (기울기 계산 X)

단계	코드	의미
1	`_, top_pred = y_pred.topk(k, 1)`	확률이 큰 순서대로 상위 k개의 **인덱스**
2	`top_pred = top_pred.t()`	전치 → (k, 배치) 형태
3	`correct = top_pred.eq(y.view(1, -1).expand_as(top_pred))`	정답과 같은지 True/False
4	`correct[:1]` 합계	1순위 예측이 맞은 개수 → acc_1
5	`correct[:k]` 합계	상위 k개 안에 정답이 있는 개수 → acc_k
6	÷ batch_size	acc_1, acc_k 반환

- correct_k: 이미지에 정확한 레이블을 부여하기 위해 사용. 즉, 첫 번째 레이블이 아닌 정확한 레이블 부여를 위해 사용 (교재)

① tensor.topk

- `torch.argmax`와 같은 효과 → 텐서에서 **가장 큰 값의 인덱스**를 얻기 위해 사용
- 즉, 네트워크 출력에서 **가장 확률이 높은 값의 인덱스**를 반환

코드	출력
`x = torch.arange(1., 6.)`	tensor([1., 2., 3., 4., 5.])
`torch.topk(x, 3)`	values = tensor([5., 4., 3.]), indices = tensor([4, 3, 2])

② t()

- 차원 0과 1을 **전치(transpose)** 하겠다는 의미

입력	`torch.t(x)` 결과
`torch.randn(3)` → tensor(0.7357) (1차원)	1차원 그대로 값 반환 → tensor(0.7357)
`torch.randn(2, 3)` → [[-0.3894, 0.6999, -0.6733], [1.7850, 0.1961, 0.2701]]	(3, 2)로 전치 → [[-0.3894, 1.7850], [0.6999, 0.1961], [-0.6733, 0.2701]]

③ 텐서 비교 함수

함수	비교
`torch.eq`	같은지
`torch.ne`	다른지

함수	비교
`torch.ge`	크거나 같은지

- 형식: `torch.eq(비교 대상 텐서, 비교할 텐서)` → 요소별로 같으면 True, 다르면 False
- 예: `torch.eq([[1, 2], [3, 4]], [[1, 1], [4, 4]])`
  - 1 vs 1 → True, 2 vs 1 → False, 3 vs 4 → False, 4 vs 4 → True
  - 결과: [[True, False], [False, True]]

> 🔎 **왜 acc_5가 항상 100%?**
> 클래스가 개·고양이 **2개뿐**인데 k=2로 상위 2개를 보면, 정답은 무조건 그 안에 있음 → top-2 정확도는 항상 100%.
> 그래서 이 예제에서 의미 있는 지표는 **acc_1(1순위 정확도)** 뿐임.

14. 학습 함수 train (코드 6-84)

- `train(model, iterator, optimizer, criterion, scheduler, device)`

순서	코드	의미
0	epoch_loss, epoch_acc_1, epoch_acc_5 = 0	누적값 초기화
1	`model.train()`	학습 모드
2	x, y `.to(device)`	배치를 장치로
3	`optimizer.zero_grad()`	기울기 초기화
4	`y_pred = model(x)`	순전파
5	`loss = criterion(y_pred[0], y)`	손실 계산
6	`calculate_topk_accuracy(y_pred[0], y)`	acc_1, acc_5
7	`loss.backward()` → `optimizer.step()`	역전파 → 가중치 업데이트
8	`+= .item()`	손실·정확도 누적
9	`/= len(iterator)`	배치 수로 나눠 평균

- epoch_acc_1: 모델이 **첫 번째로 예측한 레이블**이 붙여짐
- epoch_acc_5: 이미지에 정확한 레이블이 붙여질 것이기 때문에 **정확도가 100%** 일 것
- `y_pred[0]`인 이유: 직접 만든 ResNet의 forward가 `return x, h`로 **(최종 출력, 평탄화 특징) 두 개**를 반환하기 때문. [0] = fc를 통과한 최종 출력
- scheduler는 인자로 받기만 하고 함수 안에서 호출되지 않음 (실제로는 쓰이지 않음)

15. 평가 함수 evaluate (코드 6-85)

구분	train	evaluate
모드	`model.train()`	`model.eval()`
기울기	계산	`torch.no_grad()` → 계산 X
zero_grad / backward / step	O	X (가중치 업데이트 없음)
반환값	epoch_loss, epoch_acc_1, epoch_acc_5	동일

16. 학습 시간 측정 함수 (코드 6-86)

- `epoch_time(start_time, end_time)`
  - elapsed_time = end_time − start_time
  - elapsed_mins = int(elapsed_time / 60)
  - elapsed_secs = int(elapsed_time − elapsed_mins × 60)
  - 분, 초 반환

17. 모델 학습 (코드 6-87)

- `best_valid_loss = float('inf')` → 처음엔 무한대로 두고 더 작은 값이 나오면 갱신
- `EPOCHS = 10`
- 매 에포크
  1. `time.monotonic()`으로 시작 시간 기록
  2. `train(...)` → train_loss, train_acc_1, train_acc_5
  3. `evaluate(...)` → valid_loss, valid_acc_1, valid_acc_5
  4. `valid_loss < best_valid_loss` 이면 → 갱신 + `torch.save(model.state_dict(), '../chap06/data/ResNet-model.pt')`
  5. 종료 시간 → epoch_time → 출력

> 🧠 "검증 손실 최저 기록"을 계속 갱신하면서, 기록이 깨질 때마다 그 순간의 모델을 저장하는 것. 마지막 에포크가 아니라 **가장 좋았던 에포크의 모델**이 남음.

학습 결과

Epoch	Time	Train Loss	Train Acc @1	Valid Loss	Valid Acc @1
01	4m 4s	0.698	50.24%	0.699	51.19%
02	4m 10s	0.693	50.48%	0.699	51.19%
03	4m 13s	0.701	49.04%	0.696	47.17%
04	4m 11s	0.696	49.52%	0.698	48.96%
05	4m 2s	0.695	49.52%	0.700	50.00%
06	4m 3s	0.686	58.41%	0.693	49.85%
07	4m 7s	0.686	52.64%	0.690	51.93%
08	4m 7s	0.688	57.45%	0.693	49.55%
09	4m 0s	0.690	55.29%	0.692	54.32%
10	4m 1s	0.690	57.69%	0.693	51.93%

- Train Acc @5, Valid Acc @5는 모든 에포크에서 **100.00%**
- 오차와 정확도 측면에서 성능이 좋지 않음
  - 이미지 데이터를 늘린다면 성능은 좋아질 수 있음

- 이 예제의 목적은 성능 향상이 아닌 **CNN 관련 네트워크의 사용 방법** → 빠른 학습을 위해 데이터 개수를 제한했기 때문

> 🔎 **loss 0.693의 의미:**
> 2개 클래스를 50:50으로 찍을 때의 크로스 엔트로피 = −ln(0.5) = ln 2 ≈ **0.693**.
> 손실이 계속 0.69 근처라는 건 모델이 아직 거의 **동전 던지기** 수준이라는 뜻.

18. 테스트 데이터 예측 (코드 6-88)

- 모델이 얼마나 잘 학습되었는지 보기 위해 테스트 데이터셋 예측 결과를 **ResNet.csv**로 저장
- 테스트 이미지 하나씩 (`torch.no_grad()`)

단계	코드	의미
1	`Image.open(test_path)`	이미지 열기
2	`test_path.split('/')[-1].split('.')[1]`	파일명에서 id 추출 (예: cat.109.jpg → 109)
3	`ImageTransform(size, mean, std)` → `phase='val'`	검증용 전처리
4	`img.unsqueeze(0)`	맨 앞에 배치 차원 추가 → (1, 3, 224, 224)
5	`model.eval()` → `outputs = model(img)`	예측
6	`F.softmax(outputs[0], dim=1)[:, 1].tolist()`	소프트맥스 확률 중 **1번(개) 확률**만
7	id_list, pred_list에 추가	–

- `pd.DataFrame({'id', 'label'})` → `sort_values(by='id')` → `reset_index(drop=True)` → `to_csv('../chap06/data/ResNet.csv', index=False)` → `head(10)`

예측 결과 (ResNet.csv 상위 10개)

index	id	label
0	109	0.499019
1	145	0.528704
2	15	0.536826
3	162	0.484544
4	167	0.503171
5	200	0.585335
6	210	0.539800
7	211	0.503852
8	213	0.471151
9	224	0.524336

- 레이블이 **0.5보다 크면 개**, **0.5보다 작으면 고양이**
- 값이 전부 0.47~0.59 → 모델이 거의 확신 없이 "반반"이라고 말하는 상태
- 15가 145 뒤에 오는 이유: id가 문자열이라 **사전순 정렬**('145' < '15')

19. 예측 결과 이미지 출력 (코드 6-89)

- `classes = {0: 'cat', 1: 'dog'}`
- `display_image_grid(images_filepaths, predicted_labels=(), cols=5)`
  - rows = 이미지 수 // cols → `plt.subplots(rows, cols, figsize=(12, 6))`
  - `cv2.imread` → `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` (cv2는 BGR로 읽으므로 RGB로 변환)
  - `a = random.choice(res['id'].values)` → 해당 id의 label 값
  - label > 0.5 → 1(dog), 아니면 0(cat) → 제목으로 표시
- **그림 6-34**: 테스트 이미지 10장 중 제목이 dog 9장, cat 1장 → 고양이 사진도 대부분 dog로 표시됨

❯ 그림 6-34 테스트 데이터셋에 대한 ResNet 모델의 예측 결과



> 🔎 제목에 쓰는 label을 `random.choice`로 **무작위 id**에서 가져오기 때문에, 표시된 제목이 그 이미지 자신의 예측값과 일치하지 않음. 정확히 보려면 이미지 파일명에서 뽑은 id로 찾아야 함.

20. 정리

- 역시 정확도가 높지 않음 → 성능을 향상시키려면 **데이터를 더 늘려야** 함
- 책에서는 학습 용도로 **CNN의 다양한 모델을 어떻게 사용하는지**에 집중
  - 네트워크 위주의 학습을 위해 일부러 동일한 데이터셋 사용
  - 이 장은 CNN 다양한 모델의 **네트워크 구성 위주**로 학습하는 것을 권장
  - 성능 향상 관련해서는 **8장** 참고
- 컴퓨터 비전 분야에서 중요한 것은 모델을 학습하기 위한 **데이터**
  - 신경망은 이미 구현된 모델을 재사용할 수 있는 경우가 많음 → 누군가 만들어 놓은 신경망을 가져다 쓰면 됨
  - 중요한 점: **내가 가진 데이터에 가장 적합한 모델을 선택**하는 것
  - 이후 앞서 배운 **전이 학습**으로 약간의 튜닝만 진행하면 됨

6.2 객체 인식을 위한 신경망

1. 객체 인식(object detection)이란?

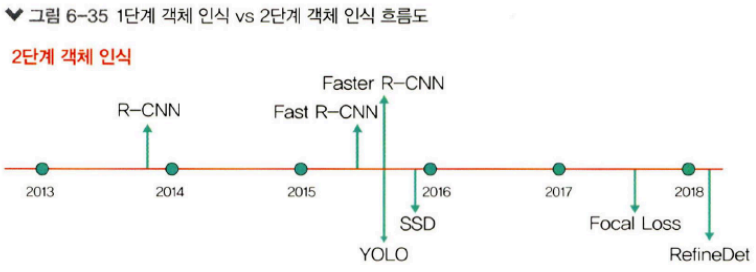
- 이미지나 영상 내에 있는 **객체를 식별**하는 컴퓨터 비전 기술
- 즉, 여러 객체에 대해
  - 각 객체가 **무엇인지 분류**하는 문제 +
  - 그 객체 위치가 **어디인지 박스(bounding box)로 나타내는 위치 검출(localization)** 문제
- **객체 인식 = 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출**
- 활용: 자율 주행 자동차, CCTV, 무인 점포 등

> 🧠 이미지 분류는 "이 사진은 고양이야"로 끝남. 객체 인식은 "고양이가 **여기(박스)** 에 있고, 사람이 **저기** 에 있어"까지 말해야 함.

2. 1단계 vs 2단계 객체 인식

구분	1단계 객체 인식 (1-stage detector)	2단계 객체 인식 (2-stage detector)
방식	분류와 위치 검출을 **동시에**	분류와 위치 검출을 **순차적으로**
속도	비교적 빠름	비교적 느림
정확도	낮음	높음
대표 모델	YOLO(You Only Look Once) 계열, SSD 계열	CNN을 처음으로 적용시킨 **R-CNN 계열**

- **그림 6-35** 흐름도 (2013~2018, 그림 기준 대략적 위치)
  - 2단계: R-CNN(2013~2014) → Fast R-CNN(2015) → Faster R-CNN(2015~2016)
  - 1단계: YOLO(2016 무렵) → SSD(2016) → Focal Loss(2017) → RefineDet(2018)
- 여기서는 **2단계 객체 인식** 알고리즘을 다룸



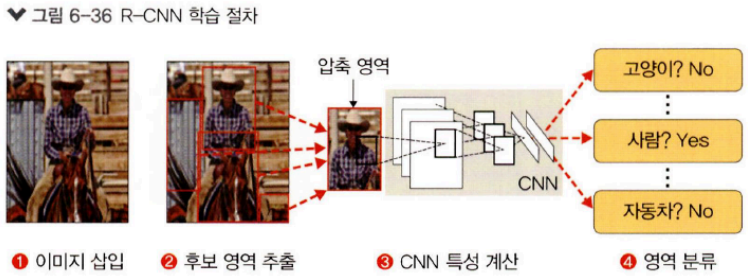
3. 6.2.1 R-CNN

3-1. 등장 배경

- 예전 객체 인식: **슬라이딩 윈도우(sliding window)** 방식
  - 일정한 크기의 윈도우(window)로 이미지의 모든 영역을 탐색하며 객체 검출
  - 이미지 왼쪽 위부터 일정 크기의 경계 상자를 만들고, 그 안에서 객체를 탐색하는 과정을 반복
  - 알고리즘의 **비효율성** 때문에 많이 사용하지 않음
- 현재: **선택적 탐색(selective search)** 알고리즘을 적용한 **후보 영역(region proposal)** 을 많이 사용
  - 후보 영역: 영상·이미지에서 객체가 있을 법한 영역

3-2. R-CNN(Region-based CNN)

- **이미지 분류를 수행하는 CNN + 객체가 있을 만한 영역을 제안해 주는 후보 영역 알고리즘**을 결합
- **그림 6-36 R-CNN 학습 절차**



단계	그림	내용
1	이미지 삽입	이미지를 입력으로 받음
2	후보 영역 추출	선택적 탐색으로 **2000개의 바운딩 박스** 추출 → 잘라 내고(cropping) → CNN에 넣기 위해 같은 크기(**227×227 픽셀**)로 통일(warping)
3	CNN 특성 계산	크기가 동일한 이미지 **2000개에 각각** CNN 모델 적용
4	영역 분류	각각 분류 → 결과 도출 (고양이? No / 사람? Yes / 자동차? No)

3-3. Note: 선택적 탐색(selective search)

- 객체 인식이나 검출을 위한 가능한 **후보 영역**(객체가 있을 만한 위치, 영역)을 알아내는 방법
- **분할 방식**으로 시드(seed)를 선정하고, 그 시드에 대한 **완전 탐색**을 적용

단계	이름	내용	그림
1단계	초기 영역 생성 (sub-segmentation)	각 객체가 영역 한 개에 할당될 수 있도록 **많은 초기 영역** 생성 = 입력 이미지를 영역 다수 개로 분할	6-37: 입력 이미지 → 분할 → 후보 영역
2단계	작은 영역의 통합	1단계에서 나눈 영역들을 비슷한 영역으로 통합. **탐욕(greedy) 알고리즘**으로 비슷한 영역이 하나로 통합될 때까지 반복	6-38: 입력 이미지 → 초기 분할 → 비슷한 영역 통합 → 비슷한 영역 통합
3단계	후보 영역 생성	2단계에서 통합된 이미지를 기반으로 후보 영역(바운딩 박스) 추출	6-39: 입력 이미지 → 비슷한 영역을 통합하여 후보 영역 생성

❯ 그림 6-37 R-CNN 학습 1단계



❯ 그림 6-38 R-CNN 학습 2단계



❯ 그림 6-39 R-CNN 학습 3단계



- 탐욕 알고리즘: 여러 경우 중 하나를 결정해야 할 때마다 **그 순간에 최적**이라고 생각되는 것을 선택해 나가는 방식

용어	의미
완전 탐색 (exhaustive search)	후보가 될 만한 대상의 크기 및 비율이 모두 다른 상황을 고려하여 후보 영역을 찾는 기법
분할 (segmentation)	영상 데이터의 특성(색상, 모양, 무늬 등)에 따라 분할하여 후보 영역을 선정하는 기법
후보 영역 (바운딩 박스)	3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
시드 (seed)	특정 기준점의 픽셀에서 점점 의미가 같은 영상 범위까지 픽셀을 확장해 나가며 분할할 때, 그 특정 기준점이 되는 픽셀

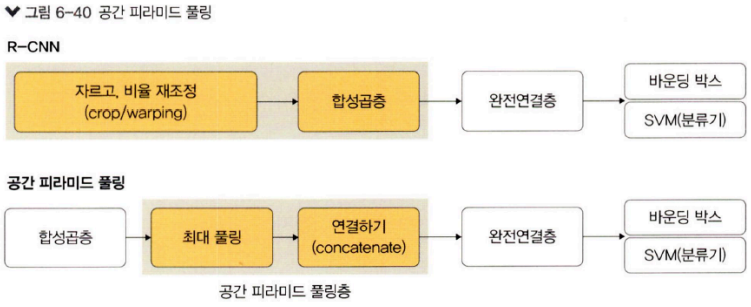
3-4. R-CNN의 단점

- 성능은 뛰어나지만 다음 단점으로 크게 발전하지 못함
  1. 앞서 언급한 **세 단계의 복잡한 학습 과정**
  2. **긴 학습 시간**과 **대용량 저장 공간**
  3. **객체 검출(object detection) 속도** 문제
- 이 문제를 해결하기 위해 **Fast R-CNN** 등장

> 🔎 가장 큰 병목은 이미지 한 장에 대해 **CNN을 2000번** 돌린다는 것. 이후 모델들은 결국 "CNN을 한 번만 돌리자"는 방향으로 개선됨.

4. 6.2.2 공간 피라미드 풀링 (spatial pyramid pooling)

- 문제: 기존 CNN 구조(예: R-CNN)는 모두 **완전연결층을 위해 입력 이미지 크기를 고정**해야 했음
  - 신경망을 통과시키려면 이미지를 고정된 크기로 **자르거나(crop) 비율을 조정(warp)** 해야 함
  - → 물체의 **일부분이 잘리거나**, 본래의 **생김새와 달라지는** 문제
- 해결: **공간 피라미드 풀링** 도입



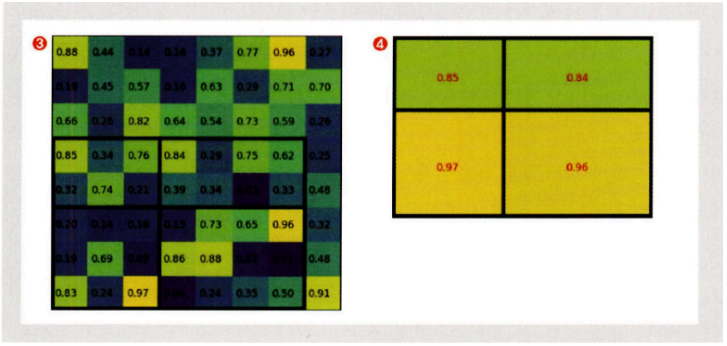
구분	흐름
R-CNN	자르고 비율 재조정(crop/warping) → 합성곱층 → 완전연결층 → 바운딩 박스 / SVM(분류기)
공간 피라미드 풀링	합성곱층 → **[최대 풀링 → 연결하기(concatenate)] = 공간 피라미드 풀링층** → 완전연결층 → 바운딩 박스 / SVM(분류기)

- 즉, **입력 이미지 크기에 관계없이** 합성곱층을 통과시키고 → 완전연결층에 전달되기 **전에** 특성 맵들을 **동일한 크기로 조절**해 주는 풀링층을 적용하는 기법
- 장점
  - 입력 이미지 크기를 조절하지 않고 합성곱층을 통과 → **원본 이미지의 특징이 훼손되지 않는** 특성 맵을 얻을 수 있음
  - 이미지 분류나 객체 인식 같은 **여러 작업에 적용** 가능

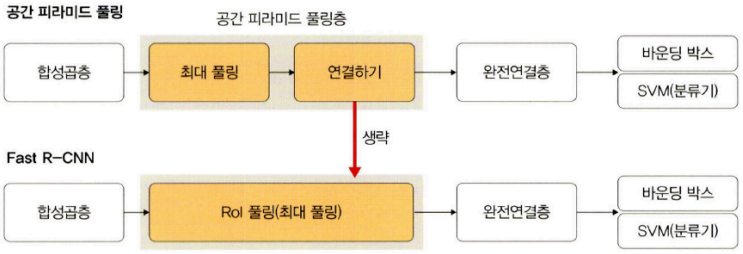
5. 6.2.3 Fast R-CNN

- R-CNN 문제: 바운딩 박스마다 CNN을 돌리고, 분류를 위한 **긴 학습 시간**
- **Fast R-CNN(Fast Region-based CNN)**: R-CNN의 속도 문제를 개선하려고 **RoI 풀링** 도입

- 선택적 탐색에서 찾은 바운딩 박스 정보가 **CNN을 통과하면서 유지**되도록 함
- 최종 CNN 특성 맵은 풀링을 적용해 완전연결층을 통과하도록 크기 조정
- → 바운딩 박스마다 CNN을 돌리는 **시간 단축**



❤ 그림 6-41 Fast R-CNN



구분	흐름
공간 피라미드 풀링	합성곱층 → [최대 풀링 → 연결하기] (공간 피라미드 풀링층) → 완전연결층 → 바운딩 박스 / SVM(분류기)
Fast R-CNN	합성곱층 → **RoI 풀링(최대 풀링)** → 완전연결층 → 바운딩 박스 / SVM(분류기)

- 공간 피라미드 풀링층의 [최대 풀링 + 연결하기]가 **생략**되고 RoI 풀링 하나로 대체됨

5-1. Note: RoI 풀링 (RoI pooling)

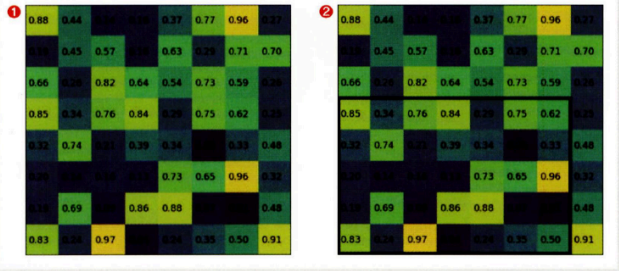
- 크기가 다른 특성 맵의 영역마다 **스트라이드를 다르게** 최대 풀링을 적용 → 결괏값 크기를 **동일하게** 맞추는 방법

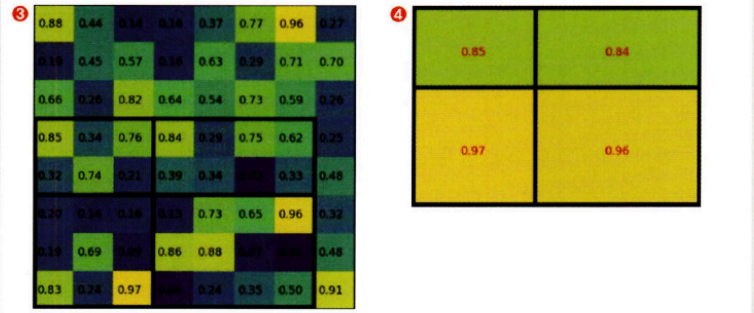
Note ≡ **RoI 풀링**

RoI 풀링(RoI pooling)은 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용하여 결괏값 크기를 동일하게 맞추는 방법입니다.

예를 들어 다음 그림과 같이 박스 한 개가 픽셀 한 개를 뜻하는 특성 맵이 있다고 합시다. 즉, 8×8 특성 맵(❶)에서 선택적 탐색으로 뽑아냈던 7×5 후보 영역(❷)이 있으며, 이것을 2×2로 만들기 위해 스트라이드(7/2=3, 5/2=2)로 풀링 영역(❸)을 정하고 최대 풀링을 적용하면 2×2 결과(❹)를 얻을 수 있습니다.

❯ 그림 6-42 RoI 풀링





단계	내용
①	8×8 특성 맵
②	선택적 탐색으로 뽑아냈던 **7×5 후보 영역**
③	2×2로 만들기 위해 스트라이드 (7/2 = 3, 5/2 = 2)로 풀링 영역을 정함
④	각 풀링 영역에 최대 풀링 → **2×2 결과**

- 2×2 결과값

	왼쪽	오른쪽
위	0.85	0.84
아래	0.97	0.96

> 🧠 후보 영역이 크든 작든 **무조건 2×2 칸으로 나눠서 칸마다 가장 큰 값 하나씩**만 뽑음. 큰 영역은 칸이 넓고 작은 영역은 칸이 좁을 뿐, 결과는 항상 2×2.

6. 세 모델 비교

구분	R-CNN	SPP	Fast R-CNN
CNN 실행 횟수	후보 영역마다 (약 2000번)	이미지 전체에 1번	이미지 전체에 1번
입력 크기 맞추기	crop/warp로 227×227 고정	크기 그대로, 공간 피라미드 풀링층에서 맞춤	크기 그대로, RoI 풀링에서 맞춤
크기 맞추는 풀링	없음	여러 격자로 최대 풀링 → 연결	한 격자로 최대 풀링
단점	느림, 저장 공간 큼, 복잡한 학습	원본 특징 보존 개선	후보 영역 추출(선택적 탐색)은 여전히 별도

6.2.4 Faster R-CNN / 6.3 이미지 분할을 위한 신경망

> 💡**Faster R-CNN**: 느린 외부 선택적 탐색 대신 **후보 영역 추출 네트워크(RPN)를 CNN 안에** 넣어서 더 빨라진 객체 인식 모델. 다양한 크기의 객체는 **앵커**(미리 정한 k개의 박스)로 대응함.
> **이미지 분할**: 이미지를 **픽셀 단위**로 분할해 객체를 추출함. 위치 정보를 지키면서(FCN) 해상도를 되살리는(역합성곱, U-Net) 방향과, 넓은 맥락을 보는(PSPNet, DeepLab) 방향으로 발전함.

1. Faster R-CNN이란?

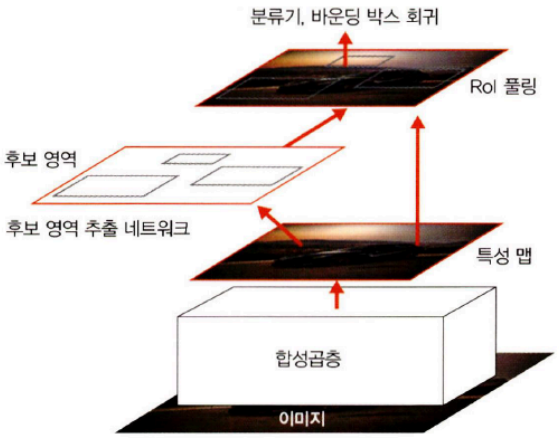
- **'더욱 빠른'** 객체 인식을 수행하기 위한 네트워크
- 기존 Fast R-CNN 속도의 걸림돌이었던 **후보 영역 생성**을 → **CNN 내부 네트워크**에서 진행하도록 설계
- 즉, **Faster R-CNN = Fast R-CNN + 후보 영역 추출 네트워크(Region Proposal Network, RPN)**

구분	Fast R-CNN	Faster R-CNN
후보 영역 생성	외부의 **선택적 탐색**	내부의 **RPN**
계산 장치	CPU로 계산 → 느림	GPU로 계산 → 빠름

2. 전체 구조

- 이미지 → **합성곱층** → **특성 맵** → **후보 영역 추출 네트워크(RPN)** → **후보 영역** → **RoI 풀링** → **분류기, 바운딩 박스 회귀**
  - RoI 풀링은 후보 영역과 함께 **특성 맵도 직접 받음**
- RPN은 **마지막 합성곱층 다음**에 위치
- 그 뒤는 Fast R-CNN과 마찬가지로 RoI 풀링, 분류기(classifier), 바운딩 박스 회귀(bounding-box regression)가 위치
- **바운딩 박스 회귀**: 바운딩 박스가 이미지의 객체를 정확히 포착하여 포함시킬 수 있도록 조정해 주는 역할

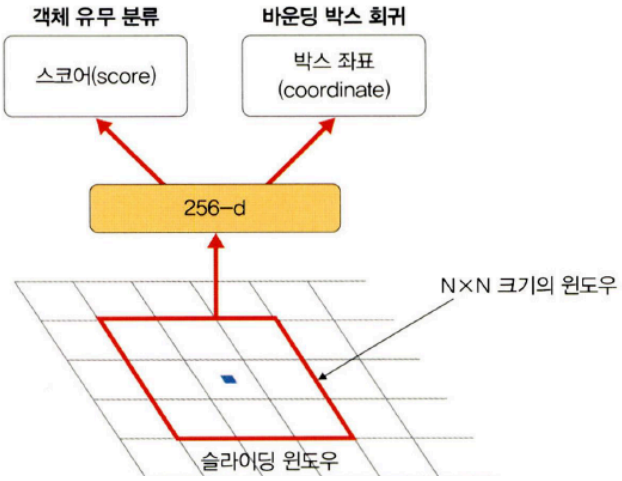
❯ 그림 6-43 Faster R-CNN



3. 후보 영역 추출 네트워크 RPN

- 특성 맵 **N×N 크기의 작은 윈도우 영역**을 입력으로 받음
- 해당 영역에 **객체의 존재 유무** 판단을 위해 **이진 분류(binary classification)** 를 수행하는 작은 네트워크를 생성
- R-CNN, Fast R-CNN에서 사용되었던 **바운딩 박스 회귀**도 위치 보정(좌표점 추론)을 위해 추가
- 하나의 특성 맵에서 모든 영역에 대한 객체 존재 유무를 확인하기 위해 → **슬라이딩 윈도우 방식**으로 N×N 윈도우를 이용하여 객체 탐색

❯ 그림 6-44 후보 영역 추출 네트워크

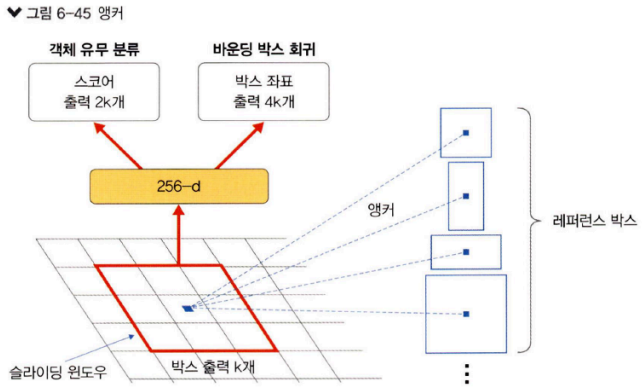


구성	내용
입력	특성 맵 위를 움직이는 N×N 크기의 윈도우 (슬라이딩 윈도우)
중간	256-d 벡터
출력 ① 객체 유무 분류	스코어(score)
출력 ② 바운딩 박스 회귀	박스 좌표(coordinate)

> 🧠 RPN은 특성 맵 위를 훑고 다니는 "탐지견". 위치마다 "여기 뭔가 있어?(예/아니요)"와 "있다면 박스를 어디쯤 쳐야 해?"만 빠르게 답해 줌. 그게 무엇인지는 뒤의 분류기가 판단함.

4. 앵커(anchor)

- 문제: 이미지에 존재하는 객체들의 **크기와 비율이 다양**함 → 고정된 N×N 입력만으로는 다양한 크기와 비율을 수용하기 어려움
- 해결: 여러 크기와 비율의 **레퍼런스 박스(reference box) k개**를 미리 정의
  - 각각의 슬라이딩 윈도우 위치마다 **박스 k개를 출력**하도록 설계
  - 이 방식을 **앵커(anchor)** 라고 함



출력	개수	의미
분류 출력 (스코어)	**2k개**	앵커마다 객체인지 배경인지 (2가지)
회귀 출력 (박스 좌표)	**4k개**	앵커마다 x, y, w, h 위치 보정 값 (4가지)
앵커 총 개수	**w × h × k개**	특성 맵 크기가 w×h일 때

5. 객체 인식 모델 총정리

구분	R-CNN	Fast R-CNN	Faster R-CNN
후보 영역	선택적 탐색 (외부, CPU)	선택적 탐색 (외부, CPU)	**RPN (내부, GPU)**
CNN 실행	후보 영역마다 (약 2000번)	이미지 전체에 1번	이미지 전체에 1번
크기 맞추기	crop/warp	RoI 풀링	RoI 풀링
핵심 개선	–	CNN 반복 제거	후보 영역 생성까지 네트워크 안으로

6.3 이미지 분할을 위한 신경망

1. 이미지 분할(image segmentation)이란?

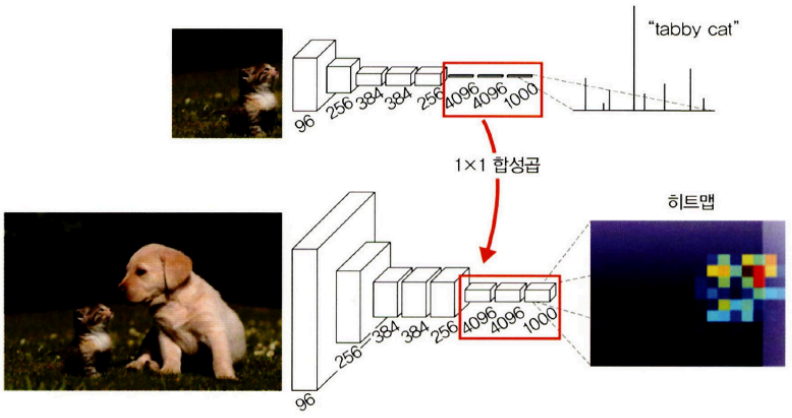
- 신경망을 훈련시켜 이미지를 **픽셀 단위로 분할**하는 것
- 즉, 이미지를 픽셀 단위로 분할하여 이미지에 포함된 **객체를 추출**

- 대표 네트워크: 완전 합성곱 네트워크, 합성곱 & 역합성곱 네트워크, U-Net, PSPNet, DeepLabv3/DeepLabv3+

6.3.1 완전 합성곱 네트워크 (FCN)

- 완전연결층의 한계
  1. **고정된 크기의 입력**만 받아들임
  2. 완전연결층을 거친 후에는 **위치 정보가 사라짐**
- 해결: 완전연결층을 **1×1 합성곱으로 대체** → 완전 합성곱 네트워크(Fully Convolutional Network, FCN)
- FCN = 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 **이미지 분할에 적합하도록** 만든 네트워크

❮ 그림 6-46 완전 합성곱 네트워크



- AlexNet 아래쪽에서 사용되었던 완전연결층 세 개(4096, 4096, 1000)를 → 1×1 합성곱으로 변환
- 위치 정보가 남아 있기 때문에 **히트맵(heatmap)** 으로 고양이의 위치를 확인할 수 있음
- 원래: 출력이 "tabby cat" 클래스 점수뿐 → 변환 후: 고양이 위치가 표시된 히트맵
- **히트맵**: 열을 뜻하는 히트(heat)와 지도를 뜻하는 맵(map)을 결합한 용어. 색상으로 표현할 수 있는 다양한 정보를 이미지 위에 열 분포 형태의 그래픽으로 출력하는 것
- 또한 합성곱층으로 사용되기 때문에 **입력 이미지 크기 제약이 사라지는** 장점

구분	완전연결층	1×1 합성곱 (FCN)
입력 크기	고정	제약 없음
위치 정보	사라짐 (일렬로 펴짐)	남아 있음 (공간 구조 유지)
출력	클래스 점수 하나	위치별 점수 → 히트맵

6.3.2 합성곱 & 역합성곱 네트워크

- FCN의 단점 (위치 정보가 보존된다는 장점에도)
  - 여러 단계의 합성곱층과 풀링층을 거치면서 **해상도가 낮아짐**

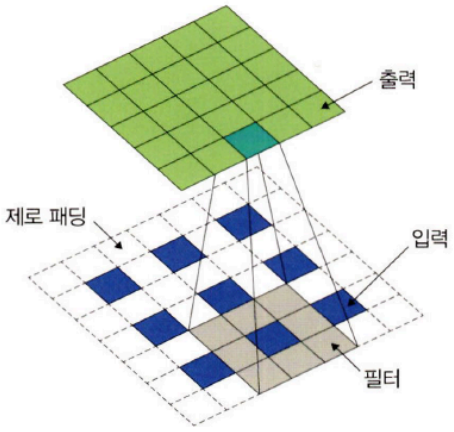
    - 낮아진 해상도를 복원하기 위해 **업 샘플링** 방식을 사용 → 이미지의 **세부 정보들을 잃어버리는** 문제
    - 업 샘플링: 최종 이미지의 크기가 입력 이미지의 크기와 같도록 하는 것
- 해결: **역합성곱 네트워크** 도입 → 합성곱 & 역합성곱 네트워크(convolutional & deconvolutional network)
- CNN의 최종 출력 결과를 **원래의 입력 이미지와 같은 크기**로 만들고 싶을 때 사용
- **시멘틱 분할(semantic segmentation)** 등에 활용 → 이미지 내에 있는 물체들을 의미 있는 단위로 분할하는 것
- 역합성곱 = **업 샘플링(upsampling)** 이라고도 함

역합성곱 (deconvolution)

구분	합성곱	역합성곱
특성 맵 크기	줄임	**증가**시킴
그림 6-47	512×512 → 256×256 → 128×128 → 64×64 → 32×32 → 1×1 (Max Pooling)	1×1 → 32×32 → 64×64 → 128×128 → 256×256 → 512×512 (Unpooling)

역합성곱 동작 방식

❯ 그림 6-48 역합성곱 진행 방식



1. 각각의 픽셀 주위에 **제로 패딩(zero-padding)** 추가
2. 이렇게 패딩된 것에 **합성곱 연산** 수행

- 그림: 아래쪽 **파란색 픽셀이 입력**, 주위의 **흰색은 제로 패딩**, **회색 필터**로 합성곱 → 위쪽 **초록색이 출력**

6.3.3 U-Net

- **바이오 메디컬 이미지 분할**을 위한 합성곱 신경망
- 메디컬 이미지 분할과 관련해서 **항상 회자되는** 네트워크

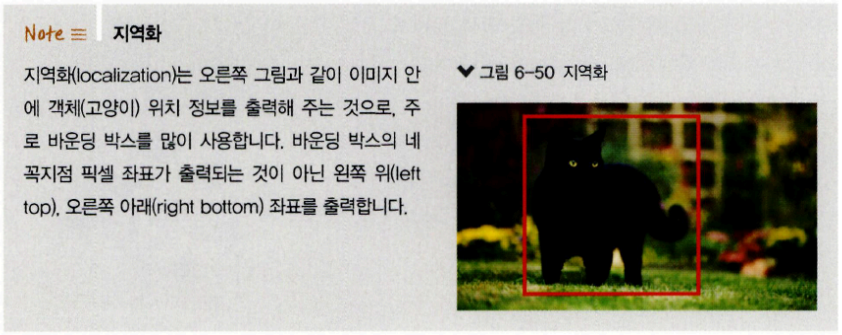
특징

특징	내용
속도가 빠르다	기존 슬라이딩 윈도우 방식은 이전 패치에서 검증이 끝난 부분을 다음 패치에서 또 검증해서 느림. U-Net은 이미 검증이 끝난 패치는 **건너뛰기** 때문에 빠름
트레이드오프에 빠지지 않는다	패치 크기가 커지면 넓은 범위를 인식해 **컨텍스트(context) 인식**에는 탁월하지만 **지역화**에는 한계. U-Net은 컨텍스트 인식과 지역화의 트레이드오프 문제를 개선

- 패치(patch): 이미지 인식 단위
- **그림 6-49**: 기존 슬라이딩 윈도우 방식은 패치 1, 2, 3이 **겹쳐 있음** → U-Net의 패치 인식 방식은 패치 1, 2, 3이 **겹치지 않고 나란히** 있음

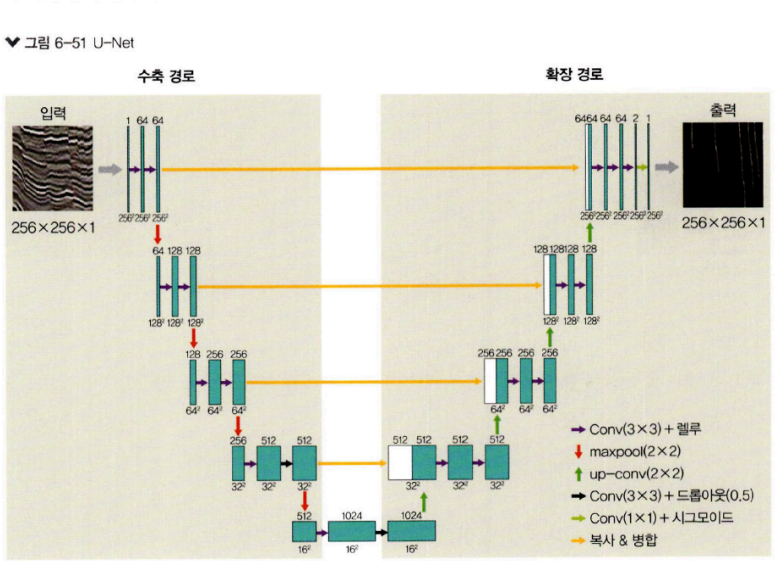
Note: 지역화(localization)

- 이미지 안에 객체(고양이) **위치 정보를 출력**해 주는 것 → 주로 **바운딩 박스** 사용
- 바운딩 박스의 네 꼭지점 픽셀 좌표가 아니라 **왼쪽 위(left top), 오른쪽 아래(right bottom)** 좌표를 출력



구조

- **FCN을 기반**으로 구축
- **수축 경로(contracting path) + 확장 경로(expansive path)**



구분	수축 경로	확장 경로
역할	**컨텍스트 포착**	특성 맵을 **업 샘플링** + 수축 경로에서 포착한 특성 맵의 컨텍스트와 **결합**해 정확한 지역화
블록	3×3 합성곱 두 개로 구성된 블록이 **네 개**	합성곱 블록 앞에 **up-conv**를 붙임
크기 변화	각 블록이 **최대 풀링(maxpool)** 으로 크기를 줄이며 다음 블록으로	수축 과정에서 줄어든 크기를 **다시 키워 가며** 합성곱 블록 이용

- U-Net은 **3×3 합성곱**이 주를 이룸 → 각 합성곱 블록은 3×3 합성곱 두 개, 그 사이에 **드롭아웃(dropout)**
- 즉, 크기가 다양한 이미지의 객체를 분할하기 위해 **크기가 다양한 특성 맵을 병합**할 수 있도록 → 다운 샘플링과 업 샘플링을 순서대로 반복하는 구조

단계	수축 경로 (채널, 크기)	확장 경로 (채널, 크기)
1	입력 256×256×1 → 64, 64 (256^2)	128 → 64, 64 → 2 → 1 (256^2) → 출력 256×256×1
2	64 → 128, 128 (128^2)	256 → 128, 128 (128^2)
3	128 → 256, 256 (64^2)	512 → 256, 256 (64^2)
4	256 → 512, 512 (32^2)	1024 → 512, 512 (32^2)
바닥	512 → 1024, 1024 (16^2)	(여기서부터 up-conv로 올라감)

그림 기호	의미
보라 화살표 →	Conv(3×3) + 렐루
빨강 화살표 ↓	maxpool(2×2)
초록 화살표 ↑	up-conv(2×2)
검정 화살표 →	Conv(3×3) + 드롭아웃(0.5)
연두 화살표 →	Conv(1×1) + 시그모이드
노랑 화살표 →	**복사 & 병합** (수축 경로 특성 맵을 확장 경로로)

> 🧠 알파벳 U 모양. 왼쪽으로 내려가며 "이게 뭔지(맥락)"를 파악하고, 오른쪽으로 올라가며 "정확히 어디인지(위치)"를 되살림. 올라갈 때 잃어버린 세부 정보는 **노란 다리(복사 & 병합)** 로 왼쪽에서 그대로 가져옴.

> 🔎 **보충**: 노란 다리는 앞에서 배운 ResNet의 숏컷과 비슷한 발상. 다만 ResNet은 **더하기**, U-Net은 채널 방향으로 **이어 붙이기(concatenate)**. 그래서 확장 경로의 채널이 예를 들어 512 + 512 = 1024처럼 두 배가 됨.

6.3.4 PSPNet

- **PSPNet(Pyramid Scene Parsing Network)**: CVPR(The IEEE Conference on Computer Vision and Pattern Recognition) **2017**에서 발표된 **시멘틱 분할** 알고리즘
- 역시 완전연결층의 한계를 극복하기 위해 **피라미드 풀링 모듈**을 추가

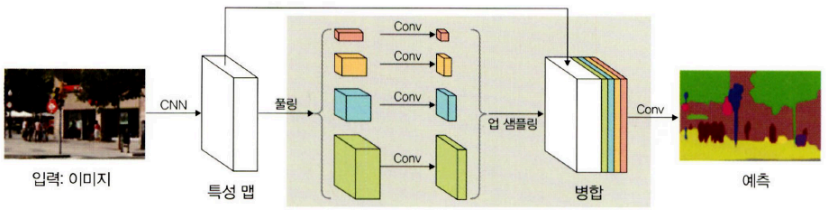
훈련 과정

단계	내용
1	이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링 → **1×1, 2×2, 3×3, 6×6** 크기로 풀링. 1×1 특성 맵이 **가장 광범위한 정보**를 담고, 각각 다른 크기의 특성 맵은 서로 다른 영역들의 정보를 담음

단계	내용
2	**1×1 합성곱**으로 채널 수 조정. 풀링층 개수가 N일 때 → 출력 채널 수 = 입력 채널 수 / N
3	모듈의 입력 크기에 맞게 특성 맵을 **업 샘플링** → 이 과정에서 **양선형 보간법(bilinear interpolation)** 사용
4	**원래의 특성 맵** + 1~3 과정에서 생성한 새로운 특성 맵들을 **병합**

- **그림 6-52**: 입력 이미지 → CNN → 특성 맵 → 풀링(4가지 크기) → 각각 Conv → 업 샘플링 → 원래 특성 맵과 병합 → Conv → 예측
- 그림은 풀링을 **네 개** 사용했지만, 구현에 따라 다르게 설정 가능

그림 6-52 PSPNet



> 🧠 같은 사진을 **멀리서(1×1), 중간 거리(2×2, 3×3), 가까이서(6×6)** 각각 본 요약을 모두 모아 함께 판단하는 것. "물 위에 있으니 이건 차가 아니라 배겠구나" 같은 **장면 전체의 맥락**을 쓸 수 있음.

> 🔎 예를 들어 입력 채널이 2048이고 풀링이 4개면, 각 갈래의 출력 채널은 2048 / 4 = 512. 병합 후 채널은 원래 2048 + 512 × 4 = 4096.

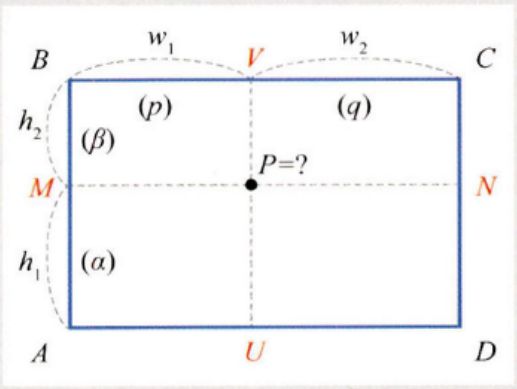
Note: 양선형 보간법 (bilinear interpolation)

- **보간법(interpolation)**: 화소 값을 할당받지 못한 영상(예: 영상의 빈 공간)은 품질이 안 좋을 수밖에 없음 → 빈 화소에 값을 할당하여 좋은 품질의 영상을 만드는 방법

구분	선형 보간법 (linear)	양선형 보간법 (bilinear)
사용 화소	원시 영상의 화소 값 **두 개**	가장 가까운 화소 **네 개**
방법	원하는 좌표에서 새로운 화소 값 계산	화소당 선형 보간을 **세 번** 수행. 네 화소에 가중치를 곱한 값을 합함

- **그림 6-53 예시**: 직사각형의 네 꼭지점 A(왼쪽 아래), B(왼쪽 위), C(오른쪽 위), D(오른쪽 아래)에 값이 주어졌을 때, 내부 임의의 점 P의 값을 추정
- P에서 x축 방향 거리 = w_1, w_2 / y축 방향 거리 = h_1, h_2
- $\alpha = h_1 / (h_1 + h_2)$, $\beta = h_2 / (h_1 + h_2)$
- $p = w_1 / (w_1 + w_2)$, $q = w_2 / (w_1 + w_2)$
- **$P = q(\beta A + \alpha B) + p(\beta D + \alpha C) = q\beta A + q\alpha B + p\beta D + p\alpha C$**

❯ 그림 6-53 양선형 보간법



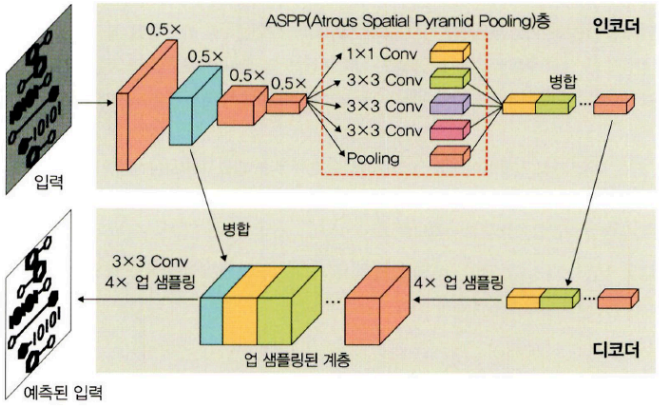
> 🧠 P에 **가까운 꼭지점일수록 더 큰 가중치**를 받음. 그래서 각 꼭지점의 가중치는 "반대쪽 거리" 비율. 예를 들어 A의 가중치 qβ는 P가 A에서 멀어질수록(오른쪽·위로 갈수록) 작아짐.
> 순서로 보면: 왼쪽 변(A, B)에서 한 번, 오른쪽 변(D, C)에서 한 번 세로로 보간한 뒤, 그 둘을 가로로 한 번 더 보간 → **선형 보간 3번**.

6.3.5 DeepLabv3 / DeepLabv3+

- 역시 완전연결층의 단점을 보완하기 위해 **Atrous 합성곱**을 사용하는 네트워크
- **인코더**와 **디코더** 구조
- 일반적인 인코더-디코더 구조에서는 불가능했던 **인코더에서 추출된 특성 맵의 해상도를** → Atrous 합성곱을 도입하여 **제어**할 수 있도록 함

인코더-디코더 구조

❯ 그림 6-54 DeepLab의 인코더-디코더 구조



구분	흐름
인코더	입력 → 0.5× → 0.5× → 0.5× → 0.5× (크기를 계속 절반으로) → **ASPP(Atrous Spatial Pyramid Pooling)층**: 1×1 Conv, 3×3 Conv, 3×3 Conv, 3×3 Conv, Pooling → 병합
디코더	인코더 출력을 **4× 업 샘플링** → 인코더 앞쪽 특성 맵과 **병합** → 3×3 Conv, **4× 업 샘플링** → 예측된 입력

> 🔎 ASPP는 PSPNet의 피라미드 풀링과 같은 아이디어를 Atrous 합성곱으로 구현한 것. 서로 다른 rate의 3×3 Atrous 합성곱을 나란히 써서 여러 범위를 동시에 봄.

Atrous 합성곱

- 필터 내부에 **빈 공간을 둔 채로** 작동
- 빈 공간의 양을 결정하는 파라미터: **rate**
  - **r = 1**: 기존 합성곱과 동일하게 빈 공간을 가짐 (빈 공간 없음)
  - r이 커질수록 빈 공간이 더 많아짐

rate	3×3 커널이 덮는 범위 (보충)	그림
r = 1	3×3	가운데 9칸이 붙어 있음
r = 2	5×5	한 칸씩 띄워서 9칸
r = 3	7×7	두 칸씩 띄워서 9칸

🔎: 덮는 범위 = r × (k − 1) + 1. 가중치는 여전히 **9개**인데 보는 범위만 넓어짐.

6-3. 왜 쓰나? — 수용 영역

- 이미지 분할에서 높은 성능을 내려면 **수용 영역(receptive field)의 크기**가 중요 → 수용 영역을 확대하면 특성을 찾는 범위가 넓어지기 때문
- Atrous 합성곱 → **파라미터 수를 늘리지 않으면서도** 수용 영역을 크게 키울 수 있음 → 이미지 분할 분야에서 많이 사용
- 일반적인 CNN: 출력이 입력의 **1/32**로 줄어듦 → Atrous 합성곱 적용: **1/8**로 줄어듦 → 특성 맵 크기가 기존 대비 **4배 보존**

❮ 그림 6-56 Atrous 합성곱 효과

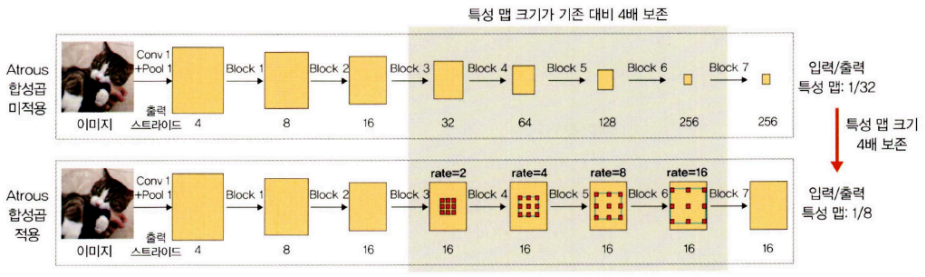


그림 6-56	Conv1+Pool1 → Block 1 → Block 2 → Block 3 → Block 4 → Block 5 → Block 6 → Block 7 (출력 스트라이드)	입력/출력 특성 맵
Atrous 합성곱 미적용	4 → 8 → 16 → 32 → 64 → 128 → 256 → 256	1/32
Atrous 합성곱 적용	4 → 8 → 16 → 16 (rate=2) → 16 (rate=4) → 16 (rate=8) → 16 (rate=16) → 16	1/8

> 🧠보통은 넓게 보려면 그림을 **축소(풀링·스트라이드)** 해야 해서 세부가 뭉개짐. Atrous는 그림 크기는 그대로 두고 **눈만 넓게 벌려서** 봄. 그래서 넓은 맥락과 높은 해상도를 동시에 챙김.

6-4. Note: 수용 영역 (receptive field)

- 외부 자극이 전체에 영향을 주는 것이 아니라 **특정 영역에만 영향**을 준다는 의미
- 영상에서 특정 위치의 픽셀들은 **주변 일부 픽셀과 연관성이 높고**, 거리가 멀어질수록 그 영향은 감소
- 영상 전체 영역에 동일한 중요도를 부여해 처리하는 대신, **특정 범위를 한정해서 처리**하여 효과적으로 훈련을 수행하는 것